MEMORANDUM

From:	Estelle Engelskirchen eengelsk@calpoly.edu	William Waite wwaite@calpoly.edu

Course:	ME 4305-S03: Mechatronics II
Date:	September 16, 2026
RE:	**Lab 0x01 – Writing Hardware Drivers**

Abstract

In this assignment, we describe our development of two Python class files, *encoder.py* and *motordriver.py*, to interface with Permanent Magnet DC (PMDC) motors and quadrature encoders. We provide a summary of our process along with excerpts from a *main.py* script designed to test the functionality of these class files.

Background

PMDC motors on the Romi robot are driven through DRV8838 H-bridge motor drivers because the Nucleo GPIO pins cannot directly supply motor power. To handle motor function, each driver uses a direction signal and PWM signal, where the GPIO pin assigns rotation direction and the PWM duty cycle produces the average voltage and requested effort of the motor. Lowering nSLEEP places the driver in coast mode, while re-enabling after commanding zero effort places it in brake mode [1, 2].

Each motor's quadrature encoder produces two digital channels that are offset by 90 degrees. The STM32 timer decodes this phase relationship so that changes in signed count indicate both motion and direction. Our encoder driver samples the timer count, accumulates count changes to determine position, and uses ticks_us() and ticks_diff() to calculate velocity in counts per second. Because the timer counter is 16-bit, it wraps between 0 and 65,535. The update() method identifies an unusually large signed count change as a rollover, corrects it by one full counter range, and adds the corrected change to the accumulated position [1, 3]. These reusable classes support the hardware tests described below.

Motor Driver Class

The *motordriver.py* class file was written to define the left and right motors as objects, and to define the following functions:

> **__init__:** Defines the direction and enable pins as objects and declares them to be output pins. Establishes the PWM pin as an object on a channel on the provided timer, sets the timer channel to PWM output, and sets the initial duty cycle to be 0.
>
> **enable:** Uses set_effort function to set the effort to 0 for "brake mode" and sets the enable pin to high to provide power to the motor.

disable: Uses enable pin object to set the pin to low and end power output.

set_effort: Accepts a numeric value as an "effort level", checks if it is between -100 and +100, and sets the direction pin either high or low to reflect the positive or negative value. In the case of our pin setup, a high direction pin corresponds to backward movement while a low direction pin corresponds to forward. The absolute value of the effort is then passed to the pulse width percent to set the motor speed according to the provided magnitude of effort.

With these definitions in place, it is now possible to use the functions and objects in this class in any file if the PWM pin, enable pin, direction pin, and timer channel are specified in the main code. This allows the class to be adaptable to any main code, regardless of which pins have been hardwired to a particular function.

Encoder Class

The *encoder.py* class file was written to define each encoder as an object, and to define the following functions:

__init__: Sets the provided timer with a period of 0xFFFF and a prescaler of 0. Defines the provided pins as the A and B channel pins required for a quadrature encoder. Sets variables used in update function and sets auto reload value at 65,535.

update: Computes delta between counts and corrects for reload at overflow using method outlined in lecture [1]. Additionally computes delta between time using ticks_us() and ticks_diff() from the timer class.

get_position: returns the most recently computed position from previous position + corrected delta.

get_velocity: returns the velocity as delta position / delta time.

zero: sets all current values to 0.

Hardware Testing Procedure and Results

The purpose of *main.py* is to call the class files *encoder.py* and *motordriver.py* to allow us to use the motors and encoders as objects after defining the correct pins and timer channels once in the program. To ensure that the class files are functioning correctly and producing the desired effect in the motors and encoders, this main code runs through a series of cases to identify the following parameters:

1. Each motor can spin forward and backward independently with varying speed.
2. Each motor can be enabled and disabled individually and does not start moving when enabled.
3. Each encoder position can grow in the positive and negative directions.
4. Timer reload has no effect on the encoder speed or position.
5. The encoder and motor objects are paired together properly in code so that spinning a motor causes its associated encoder to count.

6. The encoders each count upward when the motors have a positive duty cycle applied and the wheels rotate such that Romi would drive forward.

To apply these functions, the update function from the encoder class must be called at regular intervals during movement to identify changes in position, direction, and velocity. This was achieved with a for loop during each test case, which calls the update function for both encoders once every 10 ms for 500 iterations (5 seconds) and prints the resulting position and velocity read by each encoder every 1 second.

```
    if _ % 100 == 0: # if _ is divisible by 100, then print

      print(f"right position: {encoder_1.get_position()}") # Print RH position

      print(f"left position:  {encoder_2.get_position()}") # Print LH position

      print(f"right velocity: {encoder_1.get_velocity()}") # Print RH velocity

      print(f"left velocity:  {encoder_2.get_velocity()}") # Print LH velocity



    delay(10)
```

This allows for consistent calling of the update function to catch any overflow issues and regularly update positions and velocities. The print function allows for these values to be read and confirmed.

The following cases were tested using this for loop for each case:

Case 1: Both encoders are zeroed and enabled. Right motor turns forward while left motor turns backward at approximately the same speed.

```
1.      # Zeros the encoders at the starting position
2.      encoder_1.zero()
3.      encoder_2.zero()
4.
5.      # Enables both motors from motordriver.py
6.      left_motor.enable()
7.      right_motor.enable()
8.
9.      # Sets motor effort using pwm in motordriver.py
10.     left_motor.set_effort(-42)
11.     right_motor.set_effort(42)
```

This case produced the following output results each second during a 5 second span, as labeled.

```
MPY: sync filesystems
MPY: soft reboot
right position: 0
left position:  0
right velocity: 0.0
left velocity:  0.0
right position: 2465
left position:  -2711
right velocity: 2600.0
left velocity:  -2800.0
right position: 5034
left position:  -5538
right velocity: 2500.0
left velocity:  -2800.0
right position: 7600
left position:  -8365
right velocity: 2600.0
left velocity:  -2800.0
right position: 10162
left position:  -11200
right velocity: 2500.0
left velocity:  -2800.0
```

CASE 1

1

2

3

4

5

The first output reflects the system before the motors begin moving, so the encoders register a zero position and no velocity. One second after the first set of positions and velocities are printed, the next output shows the expected motor behavior: the right motor's position has increased (forward) with a positive velocity of 2600 counts/second and the left motor's position has decreased (backward) with a negative velocity of -2800 counts/second. The discrepancy between the position at 1 second and these velocity values is likely due to lag between encoder tick reading and the speed of the computer printing. The following four seconds illustrate the continuation of this pattern.

Case 2*:* Disable right motor, change direction of left motor.

```
# Change left motor direction and disable right motor

left_motor.set_effort(42)

right_motor.disable()
```

This code is designed to illustrate how the motors can change direction depending on the sign of the effort, and how the *disable* function works. This can be seen in the following output code, where the right motor position remains constant and the right velocity is 0. Additionally, the left motor velocity changes from negative to positive, and the left motor position grows in the positive direction.

```
right position: 13109   CASE 2
left position:  -11353
right velocity: 0.0          1
left velocity:  2800.0
-------------------------------
right position: 13109
left position:  -8434        2
right velocity: 0.0
left velocity:  2900.0
-------------------------------
right position: 13109
left position:  -5487        3
right velocity: 0.0
left velocity:  2999.7
-------------------------------
right position: 13109
left position:  -2552        4
right velocity: 0.0
left velocity:  2900.0
-------------------------------
right position: 13109
left position:  403          5
right velocity: 0.0
left velocity:  2840.909
-------------------------------
```

Case 3: Slow left motor and enable right motor without setting any effort.

```
# Change left motor speed and enable right motor

left_motor.set_effort(10)

right_motor.enable()
```

This code is meant to illustrate the ability of the *set_effort* function to slow one motor independent from the other. This can be seen in the output, where the left motor velocity drops from ~2900 ticks/s to ~600 ticks/s with an effort drop from 42 to 10.

Additionally, this code illustrates the *enable* function's ability to turn a motor on and directly enter "brake mode", where the effort is 0 and the position is held constant.

```
right position: 13109   CASE 3
left position:  1111
right velocity: 0.0          1
left velocity:  600.0
-------------------------------
right position: 13109
left position:  1715         2
right velocity: 0.0
left velocity:  600.0
-------------------------------
right position: 13109
left position:  2316         3
right velocity: 0.0
left velocity:  600.0
-------------------------------
right position: 13109
left position:  2909         4
right velocity: 0.0
left velocity:  600.0
-------------------------------
right position: 13109
left position:  3506         5
right velocity: 0.0
left velocity:  589.8545
-------------------------------
```

Case 4: Increase left motor speed and increase right motor speed to lower value.

```
# Change left motor speed and right motor speed

left_motor.set_effort(75)

right_motor.set_effort(42)
```

This code demonstrates the ability of the motors to move at different speeds and is meant to show that both motors move in the forward direction when they receive a positively signed effort value. This can be observed from the output, where the right velocity is lower than the left velocity.

```
right position: 15552   CASE 4
left position:  8324
right velocity: 2500.0      1
left velocity:  5000.0
right position: 18108
left position:  13365
right velocity: 2600.0      2
left velocity:  5000.0
right position: 20650
left position:  18361
right velocity: 2500.0      3
left velocity:  4900.0
right position: 23189
left position:  23284
right velocity: 2500.0      4
left velocity:  4800.0
right position: 25725
left position:  28325
right velocity: 2567.3944   5
left velocity:  4933.3992
```

Case 5: Disable left motor and change right motor direction.

```
# Disable left motor and change right motor speed and direction

left_motor.disable()

right_motor.set_effort(-42)
```

This case demonstrates the *disable* function on the left motor, which can be observed in the output in the constant left position and zero left velocity. Additionally, it illustrates the ability of the right motor to change direction according to the sign of the effort value, which can be seen in the negative right velocity and the decreasing right position count.

```
right position: 23287
left position:  29443
right velocity: -2600.0
left velocity:  0.0
```
CASE 5 — 1

```
right position: 20716
left position:  29443
right velocity: -2500.0
left velocity:  0.0
```
2

```
right position: 18164
left position:  29443
right velocity: -2500.0
left velocity:  0.0
```
3

```
right position: 15608
left position:  29443
right velocity: -2500.0
left velocity:  0.0
```
4

```
right position: 13061
left position:  29443
right velocity: -2531.1526
left velocity: 0.0
```
5

Case 6: Enable left motor and set high positive effort, set right motor to high negative effort for 20 seconds.

```
# Both motors on, one forward and one backward to require reload

left_motor.enable()

left_motor.set_effort(42)

right_motor.set_effort(-42)
```

This case is designed to show the *encoder.py* class ability to account for the reload of the counter at values under 0 and over 65,535. The right motor spins backward at a negative velocity for 20 seconds while the left motor spins forward at a positive velocity for 20 seconds. As shown below, the right position successfully counts down below 0 to a negative value.

```
left velocity: 2900.0
right position: 467
left position:  43726
right velocity: -2500.0
left velocity: 2900.0
right position: -2038
left position:  46645
right velocity: -2500.0
```

Similarly, the left position counter successfully counts past the overflow value of 65,535.

```
left position:  64436
right velocity: -2500.0
left velocity: 3000.0
right position: -19611
left position:  67429
```

References

[1] ME 4305, Lab 0x01 – Writing Hardware Drivers (supplied course handouts), 2026.

[2] Texas Instruments. *DRV8838 Low-Voltage H-Bridge Driver Datasheet*. https://www.ti.com/lit/ds/symlink/drv8838.pdf

[3] MicroPython.*pyb - Functions Related to the Board*. https://docs.micropython.org/en/latest/pyboard/library/pyb.html

[4] Engelskirchen, E., and W. Waite. *encoder.py*. Project source file, 2026.

[5] Engelskirchen, E., and W. Waite. *motordriver.py*. Project source file, 2026.

[6] Engelskirchen, E., and W. Waite. *main.py*. Project source file, 2026.